

Polling, WebSocket and Webhooks

1. What is Polling?

Polling is a technique where a client repeatedly requests information from a server at regular intervals to check for updates.

2. Where Polling is Used

Polling is usually used when:

1. **Real-time-ish apps** where WebSockets aren't available:
 - Notification system, Chat apps (simpler ones), Dashboard metrics (stock prices, weather, crypto prices)
2. **When server can't push updates** (stateless or behind firewalls).
3. **Short-lived data fetch**: quick updates every few seconds.

```
useEffect(() => {  
  // Polling function  
  const fetchData = async () => {  
    try {  
      const res = await fetch("/api/data");  
      const json = await res.json();  
      setData(json);  
    } catch (err) {  
      console.error(err);  
    }  
  };  
  
  // Fetch immediately  
  fetchData();  
  
  // Set interval for polling every 5 seconds  
  const interval = setInterval(fetchData, 5000);  
  
  // Cleanup on unmount  
  return () => clearInterval(interval);  
}, []);
```

Explanation:

- `fetchData()` calls the server endpoint `/api/data`.
- `setInterval(fetchData, 5000)` repeats every 5 seconds.
- `clearInterval` ensures it stops if the component unmounts.

4. Notes / Best Practices

1. **Don't poll too frequently** → increases server load.
2. If possible, use **WebSocket / SSE(Server sent events)** for real-time apps.
3. **Backoff strategy**: if request fails, wait longer before retrying.
4. Polling is **simple but less efficient**.

1. Short Polling

Definition:

The client requests data from the server **at fixed intervals**, no matter whether new data is available or not.

Implementation Example (JavaScript / React):

```
useEffect(() => {  
  const fetchData = async () => {  
    const res = await fetch("/api/messages");  
    const data = await res.json();  
    setMessages(data);  
  };  
  
  // Poll every 5 seconds  
  const interval = setInterval(fetchData, 5000);  
  
  return () => clearInterval(interval);  
}, []);
```

Characteristics:

- Requests happen **regularly**.
- Server responds immediately (even if no new data).
- Can waste bandwidth if nothing changes.
- Simple to implement.

2. Long Polling

Definition:

The client requests data, **but the server holds the request open until new data is available** or a timeout occurs. Once data is sent, the client immediately sends another request.

- **Analogy:** Asking your friend, “Tell me when a new message comes,” and they only reply when there’s actually a message.

// in backend

```
app.get("/messages", (request: Request, response: Response) => {
  const clientLength = request.query.length || 0;
  if (messages.length !== clientLength) {
    return response.status(200).json(messages);
  } // changed data send immedietly
```

```
const interval = setInterval(() => {
  if(messages.length !== clientLength){
    clearInterval(interval);
    clearTimeout(timeout);
    return response.status(200).json(messages);
  }
```

```
}, 1000); // check data changes every sec if change happen send data
```

```
const timeout=setTimeout(() => {
```

```
  clearInterval(interval);
```

```
  response.status(200).json(messages);
```

```
},30000);
```

```
}); // even after no data change send old data after 30s
```

- **This request lives for 30 seconds max in backend**
- **If data has changes since last req it send immedietly**
- **If same data keep track on data change and sends immedietly when change happen**
- **If no change for 30s send old data**

Websockets

A WebSocket is a protocol that creates a persistent two-way connection between client and server.

Unlike normal HTTP where:

Client → request

Server → response

Connection closes

WebSockets keep the connection **open**.

Client ⇌ Server

(open connection)

Both sides can **send data anytime**.

1 Install WebSocket library (backend)

In your Node server:

```
npm install ws
```

ws is the most common lightweight WebSocket library.

```
import express from "express";
```

```
import { WebSocketServer } from "ws";
```

```
const app = express();
```

```
const server = app.listen(5000, () => {
```

```
  console.log("HTTP server running on 5000");
```

```
});
```

```
const wss = new WebSocketServer({ server });
```

```
let messages: string[] = [];
```

```
wss.on("connection", (socket) => {  
  console.log("Client connected");
```

```
  socket.send(JSON.stringify(messages));
```

```
  socket.on("message", (msg) => {  
    const message = msg.toString();  
    messages.push(message);
```

```
  // broadcast to all clients
```

```
  wss.clients.forEach((client) => {  
    if (client.readyState === 1) {  
      client.send(JSON.stringify(messages));  
    }  
  });  
});
```

```
socket.on("close", () => {  
  console.log("Client disconnected");  
});  
});
```

What happens here

- 1 Client connects
- 2 Server sends current messages
- 3 Client sends message
- 4 Server broadcasts to **all connected clients**

This is **true realtime**.

3 React WebSocket Client

Modify your React app.

```
import { useEffect, useRef, useState } from "react";

function Ws() {

  const socketRef = useRef<WebSocket | null>(null);
  const [messages, setMessages] = useState<string[]>([]);
  const [input, setInput] = useState("");

  useEffect(() => {

    const socket = new WebSocket("ws://localhost:5000");
    socketRef.current = socket;

    socket.onopen = () => {
      console.log("Connected to websocket");
    };
  });
}
```

```
socket.onmessage = (event) => {  
  const data = JSON.parse(event.data);  
  setMessages(data);  
};
```

```
return () => socket.close();
```

```
}, []);
```

```
const sendMessage = () => {  
  socketRef.current?.send(input);  
  setInput("");  
};
```

```
return (
```

```
<div>
```

```
  {messages.map((m,i)=>(  
    <p key={i}>{m}</p>  
  ))}
```

```
<input
```

```
  value={input}
```

```
  onChange={(e)=>setInput(e.target.value)}
```

```
/>
```



```
<button onClick={sendMessage}>
```

```
  Send
```

```
</button>
```

```
</div>
```

```
);
```

```
}
```

```
export default Ws;
```

```
export default App;
```

WebSocket Flow

With polling:

client → request

server → response

(wait)

client → request

With WebSockets:

client ————— server
open connection

client → message

server → broadcast

server → push updates instantly

One connection stays **open permanently**.

5 Why WebSockets are better

Feature	Polling	WebSocket
Requests	many	one
Latency	delayed	instant
Server load	higher	lower
Connection	short	persistent

That's why WebSockets power:

WhatsApp, Discord, Stock trading, dashboards, Multiplayer games, Live notifications

Webhooks

A webhook is when one server automatically sends an HTTP request to another server when an event happens.

So instead of someone asking for updates, the server pushes the update.

Think of it like a phone call instead of checking voicemail repeatedly.

Normal API Flow (Pull)

Your app asks another service:

Your Server → API request → Stripe

Stripe → Response

Example:

"Did someone make a payment?"

Your server must keep asking.

Webhook Flow (Push)

Instead:

Event happens (payment success)

Stripe → POST request → your webhook endpoint

So Stripe notifies your server automatically.

1 They are just HTTP requests

Usually:

POST /webhook

Content-Type: application/json

2 They are event-driven

The request only happens when something occurs.

Example:

User pays → webhook fires

3 Your server must expose a public endpoint

Because the external service must call it.

Example:

<https://myapp.com/webhook>

4 Security is important

Most webhooks include signatures so you can verify the request is real.

Example header:

Stripe-Signature

4 Webhook vs WebSocket vs Polling

Feature	Polling	WebSocket	Webhook
Who starts communication	client	both	external server
Connection	repeated requests	persistent	event request
Realtime	delayed	instant	event triggered
Use case	dashboards	chats	service integrations

5 What We Will Build

We will simulate two services.

Service A → event generator

Service B → webhook receiver

Example event:

User created

Order placed

Payment completed

When event occurs:

Service A → POST /webhook → Service B

1 User initiates payment (normal API request)

User clicks Pay on frontend.

Frontend



POST /create-payment



Your Backend

Your backend then talks to the payment provider (Stripe, Razorpay, etc.).

Your Backend → Payment Provider API

At this point your backend usually responds to the frontend immediately.

Example response:

Payment initiated

Redirecting to payment page

So the frontend request cycle ends here.

2 Payment happens on the provider side

Now the user completes payment on Stripe/Razorpay servers.

User → Stripe payment page → payment processed

Your backend is not directly involved in this step.

3 Payment provider triggers webhook

After payment success, Stripe sends a webhook to your server.

Stripe Server



Your Backend

Payload example:

```
{
  "type": "payment.succeeded",
  "amount": 500,
  "userId": 42
}
```

Your webhook handler does things like:

- update database
- mark order paid
- send email
- trigger shipment

Example logic:

Webhook received

Update order status → PAID

Send confirmation email

How the frontend knows payment succeeded

This is the part you asked about.

There are three common methods.

Method 1 (Most Common)

Frontend polls the backend for payment status.

Flow:

Frontend → GET /order-status

Backend → "paid"

Example UI flow:

Processing payment...

Checking status...

Payment successful

Method 2 (Better UX)

Use WebSockets or SSE.

Flow:

Webhook updates database



Server pushes update via WebSocket



Frontend instantly shows "Payment Successful"

Method 3 (Redirect Flow)

Some providers redirect the user back after payment.

Stripe → redirect → /payment-success

But this redirect alone is not trusted.

The real confirmation still comes from the webhook.

Flow A — User Request

Frontend → Backend → Payment provider

Flow B — Event Notification

Payment provider → Webhook → Backend

These flows are independent.

Flow 3 Backend → frontend

Why Webhooks Exist

Without webhooks, your server would need to keep asking Stripe:

Did payment finish?

Did payment finish?

Did payment finish?

Webhooks let Stripe say:

Payment finished. Here's the event.

WEBHOOK EXPOSED BY YOUR BACKEND TO GET EVENTS UPDATES

```
app.post("/webhook", (request: Request, response: Response) => {  
  const { type, name } = request.body;  
  
  console.log(`Webhook Recieved: Type :- ${type} , Name: ${name}`);  
  
  response.status(200).send("Webhook recieved");  
});
```


A SERVICE WHICH USE WEBHOOK TO POST EVENT

```
const users: String[] = [];
```

```
app.post("/users", async (request: Request, response: Response) => {  
  const { name } = request.body;
```

```
  users.push(name);
```

```
  await fetch("http://localhost:5000/webhook", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json",  
    },  
    body: JSON.stringify({ type: "User.created", name }),  
  });  
});
```